Stat_4210_Chapter10_Lab

March 11, 2026

[143]:
```
from functools import partial
print = partial(print, flush=True)
```

[144]:
```
# from google.colab import drive
# drive.mount('/content/drive/')
```

[145]:
```
# %cd /content/drive/MyDrive/Stat_4210/
```

[146]:
```
import numpy as np, pandas as pd
from matplotlib.pyplot import subplots
from sklearn.linear_model import LinearRegression, LogisticRegression, Lasso
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import KFold
from sklearn.pipeline import Pipeline
from ISLP import load_data
from ISLP.models import ModelSpec as MS
from sklearn.model_selection import train_test_split, GridSearchCV
```

[147]:
```
import torch
from torch import nn
from torch.optim import RMSprop
from torch.utils.data import TensorDataset

device = torch.device(
    "cuda" if torch.cuda.is_available()
    else "mps" if torch.backends.mps.is_available()
    else "cpu"
)

print(f"Using device: {device}")
```

```
Using device: cuda
```

[148]:
```
from torchmetrics import MeanAbsoluteError, R2Score
from torchinfo import summary
```

[149]:
```
from pytorch_lightning import Trainer
from pytorch_lightning.loggers import CSVLogger
```

[150]:
```
from pytorch_lightning import seed_everything
seed_everything(0, workers=True)
torch.use_deterministic_algorithms(True, warn_only=True)
```

```
Seed set to 0
```

[151]:
```
from torchvision.io import read_image
from torchvision.datasets import MNIST, CIFAR100
from torchvision.models import resnet50, ResNet50_Weights
from torchvision.transforms import Resize, Normalize, CenterCrop, ToTensor
```

[152]:
```
from ISLP.torch import SimpleDataModule, SimpleModule, ErrorTracker,␣
↪rec_num_workers
```

[153]:
```
from ISLP.torch.imdb import load_lookup, load_tensor, load_sparse,␣
↪load_sequential
```

[154]:
```
from glob import glob
import json
```

[155]:
```
Hitters = load_data('Hitters').dropna()
n = Hitters.shape[0]
```

[121]:
```
model = MS(Hitters.columns.drop('Salary'), intercept=False)
X = model.fit_transform(Hitters).to_numpy()
Y = Hitters['Salary'].to_numpy()
```

[122]:
```
(X_train, X_test, Y_train, Y_test) = train_test_split(X, Y, test_size=1/3,␣
↪random_state=1)
```

[123]:
```
hit_lm = LinearRegression().fit(X_train, Y_train)
Yhat_test = hit_lm.predict(X_test)
np.abs(Yhat_test - Y_test).mean()
```

[123]:
```
np.float64(259.71528833146294)
```

[124]:
```
scaler = StandardScaler(with_mean=True, with_std=True)
lasso = Lasso(warm_start=True, max_iter=30000)
standard_lasso = Pipeline(steps=[('scaler', scaler), ('lasso', lasso)])
```

[125]:
```
X_s = scaler.fit_transform(X_train)
n = X_s.shape[0]
lam_max = np.fabs(X_s.T.dot(Y_train - Y_train.mean())).max() / n
param_grid = {'lasso__alpha': np.exp(np.linspace(0, np.log(0.01), 100)) *␣
↪lam_max}
```

[126]:
```
cv = KFold(10, shuffle=True, random_state=1)
```

```
grid = GridSearchCV(standard_lasso, param_grid, cv=cv,␣
 ↪scoring='neg_mean_absolute_error')
grid.fit(X_train, Y_train);
```

[127]:
```
trained_lasso = grid.best_estimator_
Yhat_test = trained_lasso.predict(X_test)
np.fabs(Yhat_test - Y_test).mean()
```

[127]:
```
np.float64(235.67548374780287)
```

[128]:
```
class HittersModel(nn.Module):
    def __init__(self, input_size):
        super(HittersModel, self).__init__()
        self.flatten = nn.Flatten()
        self.sequential = nn.Sequential(
            nn.Linear(input_size, 50),
            nn.ReLU(),
            nn.Dropout(0.4),
            nn.Linear(50, 1))

    def forward(self, x):
        x = self.flatten(x)
        return torch.flatten(self.sequential(x))
```

[129]:
```
hit_model = HittersModel(X.shape[1])
```

[130]:
```
summary(hit_model, input_size=X_train.shape, col_names=['input_size',␣
 ↪'output_size', 'num_params'])
```

[130]:
```
==========================================================================
==================================
Layer (type:depth-idx)                   Input Shape               Output Shape
Param #
==========================================================================
==================================
HittersModel                             [175, 19]                 [175]
--
 Flatten: 1-1                            [175, 19]                 [175, 19]
--
 Sequential: 1-2                         [175, 19]                 [175, 1]
--
      Linear: 2-1                        [175, 19]                 [175, 50]
1,000
      ReLU: 2-2                          [175, 50]                 [175, 50]
--
      Dropout: 2-3                       [175, 50]                 [175, 50]
--
```

```
     Linear: 2-4                        [175, 50]                 [175, 1]
51
==========================================================================
==================================
Total params: 1,051
Trainable params: 1,051
Non-trainable params: 0
Total mult-adds (Units.MEGABYTES): 0.18
==========================================================================
==================================
Input size (MB): 0.01
Forward/backward pass size (MB): 0.07
Params size (MB): 0.00
Estimated Total Size (MB): 0.09
==========================================================================
==================================
```

[131]:
```
X_train_t = torch.tensor(X_train.astype(np.float32))
Y_train_t = torch.tensor(Y_train.astype(np.float32))
hit_train = TensorDataset(X_train_t, Y_train_t)
```

[132]:
```
X_test_t = torch.tensor(X_test.astype(np.float32))
Y_test_t = torch.tensor(Y_test.astype(np.float32))
hit_test = TensorDataset(X_test_t, Y_test_t)
```

[133]:
```
max_num_workers = rec_num_workers()
```

[134]:
```
hit_dm = SimpleDataModule(hit_train, hit_test, batch_size=32,␣
 ↪num_workers=min(4, max_num_workers), validation=hit_test)
```

[135]:
```
hit_module = SimpleModule.regression(hit_model, metrics={'mae':
 ↪MeanAbsoluteError()})
```

[136]:
```
hit_logger = CSVLogger('logs', name='hitters')
```

[137]:
```
hit_trainer = Trainer(deterministic=True,
                      max_epochs=50,
                      log_every_n_steps=5,
                      logger=hit_logger,
                      callbacks=[ErrorTracker()])
hit_trainer.fit(hit_module, datamodule=hit_dm)
```

```
 Tip: For seamless cloud uploads and versioning, try installing
[litmodels](https://pypi.org/project/litmodels/) to enable LitModelCheckpoint,
which syncs automatically with the Lightning model registry.
GPU available: True (cuda), used: True
TPU available: False, using: 0 TPU cores
LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]
```

```
  | Name  | Type         | Params | Mode  | FLOPs
-----------------------------------------------------
0 | model | HittersModel | 1.1 K  | train | 0
1 | loss  | MSELoss      | 0      | train | 0
-----------------------------------------------------
1.1 K     Trainable params
0         Non-trainable params
1.1 K     Total params
0.004     Total estimated model params size (MB)
8         Modules in train mode
0         Modules in eval mode
0         Total Flops
```

```
Sanity Checking: |          | 0/? [00:00<?, ?it/s]
```

```
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\pytorch_lightning\utilities\_pytree.py:21: `isinstance(treespec,
LeafSpec)` is deprecated, use `isinstance(treespec, TreeSpec) and
treespec.is_leaf()` instead.
```

```
Epoch 49: 100%|      | 6/6 [00:00<00:00, 117.82it/s, v_num=1]
```

```
`Trainer.fit` stopped: `max_epochs=50` reached.
```

```
Epoch 49: 100%|      | 6/6 [00:00<00:00, 104.93it/s, v_num=1]
```

[138]:
```
hit_trainer.test(hit_module, datamodule=hit_dm)
```

```
LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]
```

```
Testing DataLoader 0: 100%|      | 3/3 [00:00<00:00, 175.75it/s]
```

Test metric	DataLoader 0
test_loss	123543.3203125
test_mae	248.27793884277344

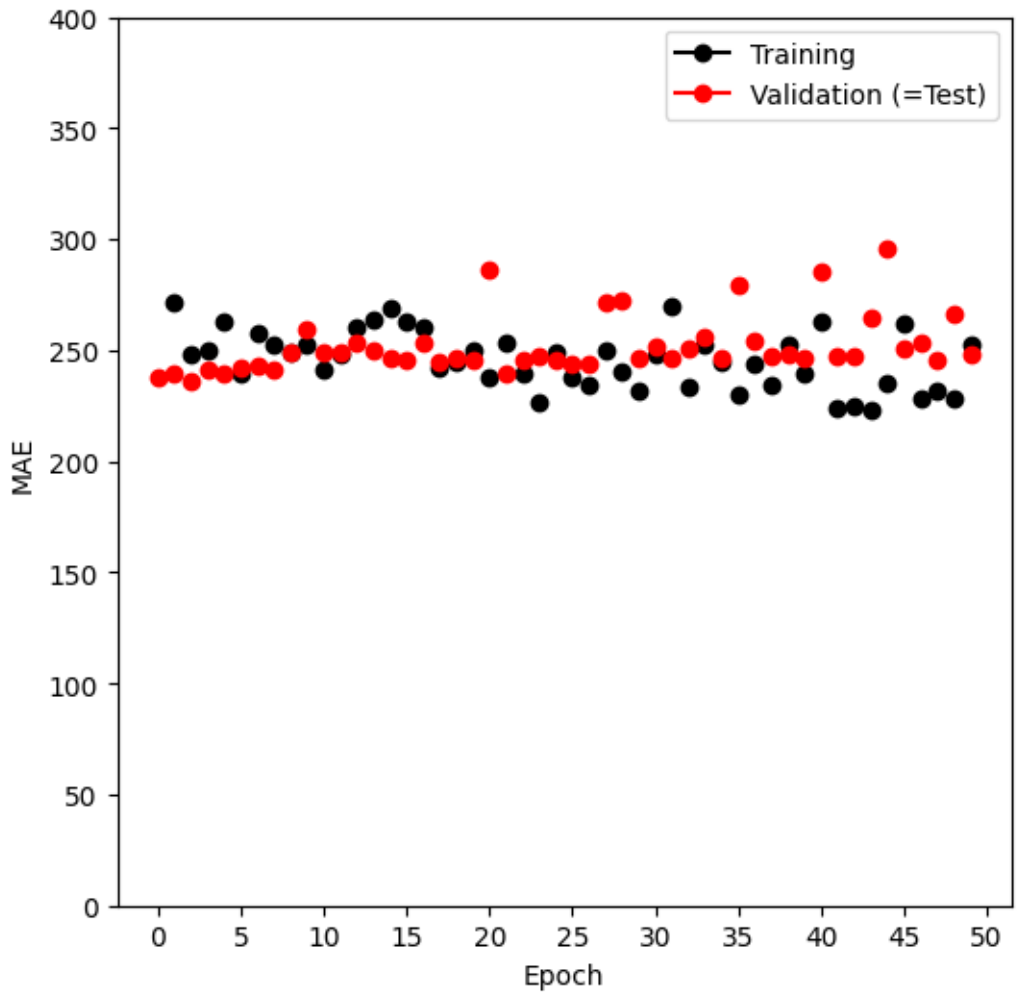
[138]:
```
[{'test_loss': 123543.3203125, 'test_mae': 248.27793884277344}]
```

[139]:
```
hit_results = pd.read_csv(hit_logger.experiment.metrics_file_path)
```

[140]:
```
def summary_plot(results, ax, col='loss', valid_legend='Validation',␣
↪training_legend='Training', ylabel='Loss', fontsize=20):
    for (column, color, label) in zip([f'train_{col}_epoch', f'valid_{col}'],␣
↪['black', 'red'], [training_legend, valid_legend]):
```

```
        results.plot(x='epoch', y=column, label=label, marker='o', color=color,␣
↪ax=ax)
    ax.set_xlabel('Epoch')
    ax.set_ylabel(ylabel)
    return ax
```

[141]:
```
fig, ax = subplots(1, 1, figsize=(6, 6))
ax = summary_plot(hit_results, ax, col='mae', ylabel='MAE',␣
↪valid_legend='Validation (=Test)')
ax.set_ylim([0, 400])
ax.set_xticks(np.linspace(0, 50, 11).astype(int));
```



[156]:
```
hit_model.eval()
preds = hit_module(X_test_t)
torch.abs(Y_test_t - preds).mean()
```

[156]: tensor(248.2779, grad_fn=<MeanBackward0>)

[157]:
```
del(Hitters,
hit_model, hit_dm,
hit_logger,
hit_test, hit_train,
X, Y,
X_test, X_train,
Y_test, Y_train,
X_test_t, Y_test_t,
hit_trainer, hit_module)
```

[158]:
```
(mnist_train, mnist_test) = [MNIST(root='data', train=train, download=True,␣
↪transform=ToTensor()) for train in [True, False]]
mnist_train
```

```
100%|      | 9.91M/9.91M [00:00<00:00, 21.4MB/s]
100%|      | 28.9k/28.9k [00:00<00:00, 687kB/s]
100%|      | 1.65M/1.65M [00:00<00:00, 7.07MB/s]
100%|      | 4.54k/4.54k [00:00<00:00, 20.7MB/s]
```

[158]:
```
Dataset MNIST
    Number of datapoints: 60000
    Root location: data
    Split: Train
    StandardTransform
Transform: ToTensor()
```

[159]:
```
mnist_dm = SimpleDataModule(mnist_train, mnist_test, validation=0.2,␣
↪num_workers=max_num_workers, batch_size=256)
```

[160]:
```
for idx, (X_ ,Y_) in enumerate(mnist_dm.train_dataloader()):
    print('X: ', X_.shape)
    print('Y: ', Y_.shape)
    if idx >= 1:
        break
```

```
X:  torch.Size([256, 1, 28, 28])
Y:  torch.Size([256])
X:  torch.Size([256, 1, 28, 28])
Y:  torch.Size([256])
```

[161]:
```
class MNISTModel(nn.Module):
    def __init__(self):
        super(MNISTModel, self).__init__()
        self.layer1 = nn.Sequential(
            nn.Flatten(),
            nn.Linear(28*28, 256),
```

```
            nn.ReLU(),
            nn.Dropout(0.4))
        self.layer2 = nn.Sequential(
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Dropout(0.3))
        self._forward = nn.Sequential(
            self.layer1,
            self.layer2,
            nn.Linear(128, 10))
    def forward(self, x):
        return self._forward(x)
```

[162]:
```
mnist_model = MNISTModel()
```

[163]:
```
mnist_model(X_).size()
```

[163]:
```
torch.Size([256, 10])
```

[164]:
```
summary(mnist_model, input_data=X_, col_names=['input_size', 'output_size',␣
↪'num_params'])
```

[164]:
```
==========================================================================================
==================================
Layer (type:depth-idx)                   Input Shape               Output Shape
Param #
==========================================================================================
==================================
MNISTModel                               [256, 1, 28, 28]          [256, 10]
--
 Sequential: 1-1                         [256, 1, 28, 28]          [256, 10]
--
      Sequential: 2-1                    [256, 1, 28, 28]          [256, 256]
--
           Flatten: 3-1                  [256, 1, 28, 28]          [256, 784]
--
           Linear: 3-2                   [256, 784]                [256, 256]
200,960
           ReLU: 3-3                     [256, 256]                [256, 256]
--
           Dropout: 3-4                  [256, 256]                [256, 256]
--
      Sequential: 2-2                    [256, 256]                [256, 128]
--
           Linear: 3-5                   [256, 256]                [256, 128]
32,896
           ReLU: 3-6                     [256, 128]                [256, 128]
```

```
--
          Dropout: 3-7                    [256, 128]                [256, 128]
--
     Linear: 2-3                          [256, 128]                [256, 10]
1,290
==========================================================================================
==================================
Total params: 235,146
Trainable params: 235,146
Non-trainable params: 0
Total mult-adds (Units.MEGABYTES): 60.20
==========================================================================================
==================================
Input size (MB): 0.80
Forward/backward pass size (MB): 0.81
Params size (MB): 0.94
Estimated Total Size (MB): 2.55
==========================================================================================
==================================
```

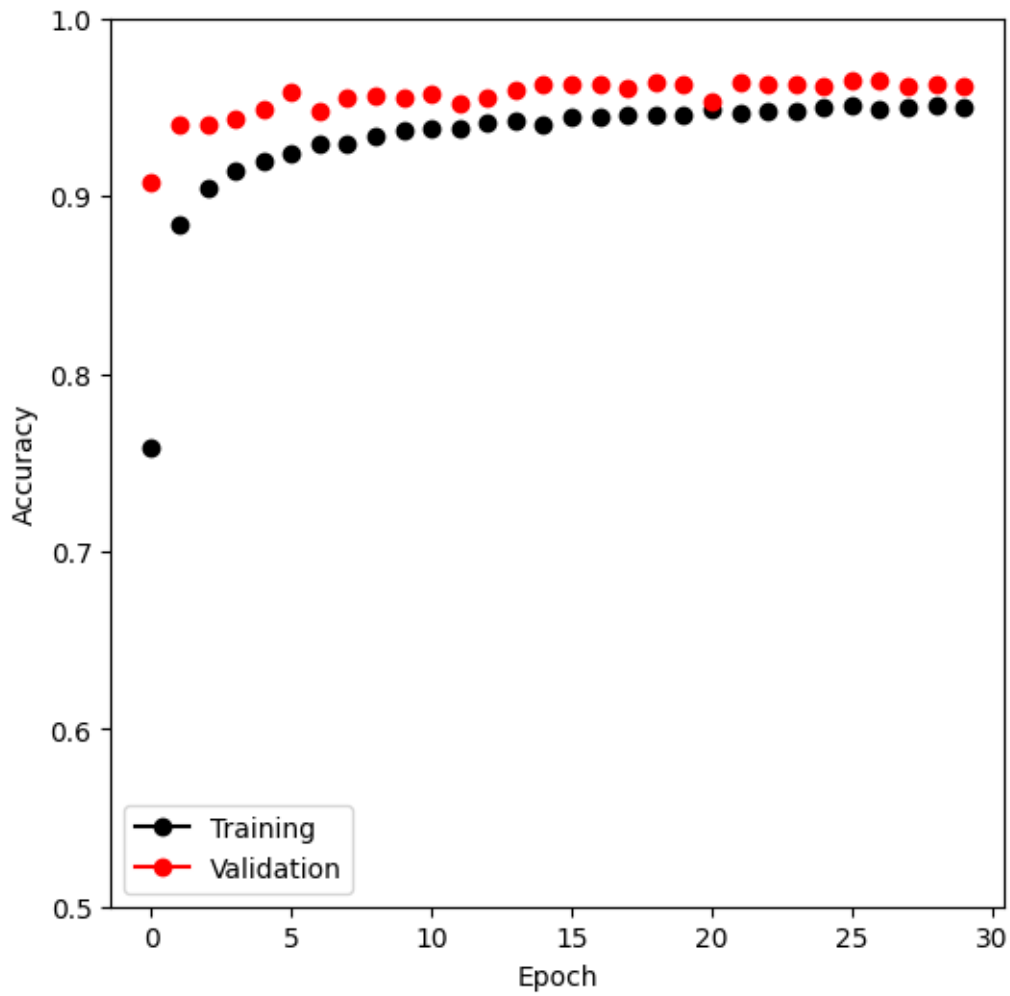
[165]:
```
mnist_module = SimpleModule.classification(mnist_model, num_classes=10)
mnist_logger = CSVLogger('logs', name='MNIST')
```

[166]:
```
mnist_trainer = Trainer(deterministic=True, max_epochs=30, logger=mnist_logger,␣
↪enable_progress_bar=False, callbacks=[ErrorTracker()])
mnist_trainer.fit(mnist_module, datamodule=mnist_dm)
```

```
 Tip: For seamless cloud uploads and versioning, try installing
[litmodels](https://pypi.org/project/litmodels/) to enable LitModelCheckpoint,
which syncs automatically with the Lightning model registry.
GPU available: True (cuda), used: True
TPU available: False, using: 0 TPU cores
LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]

  | Name  | Type             | Params | Mode  | FLOPs
-----------------------------------------------------------
0 | model | MNISTModel       | 235 K  | train | 0
1 | loss  | CrossEntropyLoss | 0      | train | 0
-----------------------------------------------------------
235 K     Trainable params
0         Non-trainable params
235 K     Total params
0.941     Total estimated model params size (MB)
13        Modules in train mode
0         Modules in eval mode
0         Total Flops
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\pytorch_lightning\utilities\_pytree.py:21: `isinstance(treespec,
```

```
LeafSpec)` is deprecated, use `isinstance(treespec, TreeSpec) and
treespec.is_leaf()` instead.
`Trainer.fit` stopped: `max_epochs=30` reached.
```

[167]:
```
mnist_results = pd.read_csv(mnist_logger.experiment.metrics_file_path)
fig, ax = subplots(1, 1, figsize=(6, 6))
summary_plot(mnist_results,
             ax,
             col='accuracy',
             ylabel='Accuracy')
ax.set_ylim([0.5, 1])
ax.set_ylabel('Accuracy')
ax.set_xticks(np.linspace(0, 30, 7).astype(int));
```



[168]:
```
mnist_trainer.test(mnist_module, datamodule=mnist_dm)
```

```
LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]
```

```
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\pytorch_lightning\utilities\_pytree.py:21: `isinstance(treespec,
LeafSpec)` is deprecated, use `isinstance(treespec, TreeSpec) and
treespec.is_leaf()` instead.


       Test metric             DataLoader 0


      test_accuracy          0.964900016784668
        test_loss           0.13647891581058502
```

[168]:
```
[{'test_loss': 0.13647891581058502, 'test_accuracy': 0.964900016784668}]
```

[170]:
```python
class MNIST_MLR(nn.Module):
    def __init__(self):
        super(MNIST_MLR, self).__init__()
        self.linear = nn.Sequential(nn.Flatten(),
                                    nn.Linear(784, 10))
    def forward(self, x):
        return self.linear(x)

mlr_model = MNIST_MLR()
mlr_module = SimpleModule.classification(mlr_model,
                                         num_classes=10)
mlr_logger = CSVLogger('logs', name='MNIST_MLR')
```

[171]:
```python
mlr_trainer = Trainer(deterministic=True,
                      max_epochs=30,
                      enable_progress_bar=False,
                      callbacks=[ErrorTracker()])
mlr_trainer.fit(mlr_module, datamodule=mnist_dm)
```

```
 Tip: For seamless cloud uploads and versioning, try installing
[litmodels](https://pypi.org/project/litmodels/) to enable LitModelCheckpoint,
which syncs automatically with the Lightning model registry.
GPU available: True (cuda), used: True
TPU available: False, using: 0 TPU cores
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-packages\pytorch_l
ightning\trainer\connectors\logger_connector\logger_connector.py:76: Starting
from v1.9.0, `tensorboardX` has been removed as a dependency of the
`pytorch_lightning` package, due to potential conflicts with other packages in
the ML ecosystem. For this reason, `logger=True` will use `CSVLogger` as the
default logger, unless the `tensorboard` or `tensorboardX` packages are found.
Please `pip install lightning[extra]` or one of them to enable TensorBoard
support by default
```

```
LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]

  | Name  | Type             | Params | Mode  | FLOPs
-----------------------------------------------------------
0 | model | MNIST_MLR        | 7.9 K  | train | 0
1 | loss  | CrossEntropyLoss | 0      | train | 0
-----------------------------------------------------------
7.9 K     Trainable params
0         Non-trainable params
7.9 K     Total params
0.031     Total estimated model params size (MB)
5         Modules in train mode
0         Modules in eval mode
0         Total Flops
`Trainer.fit` stopped: `max_epochs=30` reached.
```

[172]:
```python
mlr_trainer.test(mlr_module, datamodule=mnist_dm)
```

```
LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]


       Test metric             DataLoader 0


      test_accuracy         0.9174000024795532
        test_loss           0.33102861046791077
```

[172]:
```
[{'test_loss': 0.33102861046791077, 'test_accuracy': 0.9174000024795532}]
```

[173]:
```python
del(mnist_test,
    mnist_train,
    mnist_model,
    mnist_dm,
    mnist_trainer,
    mnist_module,
    mnist_results,
    mlr_model,
    mlr_module,
    mlr_trainer)
```

[ ]:
```python
(cifar_train, cifar_test) = [CIFAR100(root="data", train=train, download=True)␣
 ↪for train in [True, False]]
```

```
100%|      | 169M/169M [00:03<00:00, 52.5MB/s]
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\torchvision\datasets\cifar.py:83: VisibleDeprecationWarning: dtype():
```

```
align should be passed as Python or NumPy boolean but got `align=0`. Did you
mean to pass a tuple to create a subarray type? (Deprecated NumPy 2.4)
  entry = pickle.load(f, encoding="latin1")
```

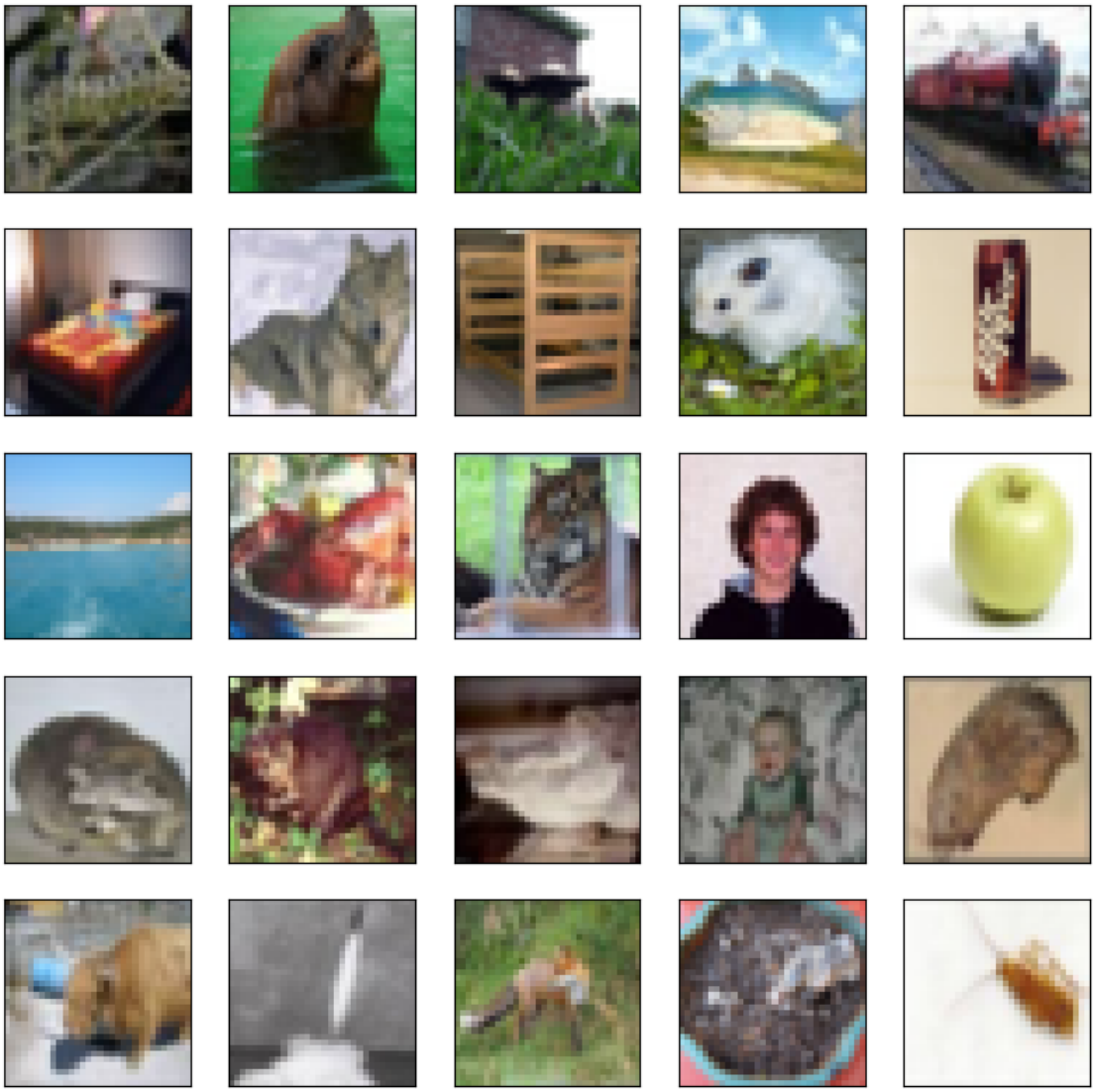
[175]:
```
transform = ToTensor()
cifar_train_X = torch.stack([transform(x) for x in cifar_train.data])
cifar_test_X = torch.stack([transform(x) for x in cifar_test.data])
cifar_train = TensorDataset(cifar_train_X, torch.tensor(cifar_train.targets))
cifar_test = TensorDataset(cifar_test_X, torch.tensor(cifar_test.targets))
```

[176]:
```
cifar_dm = SimpleDataModule(cifar_train, cifar_test, validation=0.2,␣
 ↪num_workers=max_num_workers, batch_size=128)
```

[177]:
```
for idx, (X_ ,Y_) in enumerate(cifar_dm.train_dataloader()):
    print('X: ', X_.shape)
    print('Y: ', Y_.shape)
    if idx >= 1:
        break
```

```
X:  torch.Size([128, 3, 32, 32])
Y:  torch.Size([128])
X:  torch.Size([128, 3, 32, 32])
Y:  torch.Size([128])
```

[179]:
```
fig, axes = subplots(5, 5, figsize=(10,10))
rng = np.random.default_rng(4)
indices = rng.choice(np.arange(len(cifar_train)), 25,
                     replace=False).reshape((5,5))
for i in range(5):
    for j in range(5):
        idx = indices[i,j]
        axes[i,j].imshow(np.transpose(cifar_train[idx][0], [1,2,0]),␣
 ↪interpolation=None)
        axes[i,j].set_xticks([])
        axes[i,j].set_yticks([])
```



[180]:
```
class BuildingBlock(nn.Module):

    def __init__(self, in_channels, out_channels):

        super(BuildingBlock, self).__init__()
        self.conv = nn.Conv2d(in_channels=in_channels,␣
↪out_channels=out_channels, kernel_size=(3,3), padding='same')
        self.activation = nn.ReLU()
        self.pool = nn.MaxPool2d(kernel_size=(2,2))

    def forward(self, x):
        return self.pool(self.activation(self.conv(x)))
```

[181]:
```
class CIFARModel(nn.Module):

    def __init__(self):
        super(CIFARModel, self).__init__()
        sizes = [(3,32), (32,64), (64,128), (128,256)]
        self.conv = nn.Sequential(*[BuildingBlock(in_, out_) for in_, out_ in␣
↪sizes])

        self.output = nn.Sequential(nn.Dropout(0.5), nn.Linear(2*2*256, 512),␣
↪nn.ReLU(), nn.Linear(512, 100))
    def forward(self, x):
        val = self.conv(x)
        val = torch.flatten(val, start_dim=1)
        return self.output(val)
```

[182]:
```
cifar_model = CIFARModel()
summary(cifar_model,
        input_data=X_,
        col_names=['input_size', 'output_size', 'num_params'])
```

[182]:
```
==============================================================================
==================================
Layer (type:depth-idx)                   Input Shape               Output Shape
Param #
==============================================================================
==================================
CIFARModel                               [128, 3, 32, 32]          [128, 100]
--
 Sequential: 1-1                         [128, 3, 32, 32]          [128, 256, 2,
2]          --
     BuildingBlock: 2-1                  [128, 3, 32, 32]          [128, 32, 16,
16]         --
          Conv2d: 3-1                    [128, 3, 32, 32]          [128, 32, 32,
32]         896
          ReLU: 3-2                      [128, 32, 32, 32]         [128, 32, 32,
32]         --
          MaxPool2d: 3-3                 [128, 32, 32, 32]         [128, 32, 16,
16]         --
     BuildingBlock: 2-2                  [128, 32, 16, 16]         [128, 64, 8,
8]           --
          Conv2d: 3-4                    [128, 32, 16, 16]         [128, 64, 16,
16]         18,496
          ReLU: 3-5                      [128, 64, 16, 16]         [128, 64, 16,
16]         --
          MaxPool2d: 3-6                 [128, 64, 16, 16]         [128, 64, 8,
8]           --
     BuildingBlock: 2-3                  [128, 64, 8, 8]           [128, 128, 4,
```

```
4]          --
          Conv2d: 3-7                    [128, 64, 8, 8]           [128, 128, 8,
8]          73,856
          ReLU: 3-8                      [128, 128, 8, 8]          [128, 128, 8,
8]          --
          MaxPool2d: 3-9                 [128, 128, 8, 8]          [128, 128, 4,
4]          --
     BuildingBlock: 2-4                   [128, 128, 4, 4]          [128, 256, 2,
2]          --
          Conv2d: 3-10                   [128, 128, 4, 4]          [128, 256, 4,
4]          295,168
          ReLU: 3-11                     [128, 256, 4, 4]          [128, 256, 4,
4]          --
          MaxPool2d: 3-12                [128, 256, 4, 4]          [128, 256, 2,
2]          --
 Sequential: 1-2                          [128, 1024]               [128, 100]
--
     Dropout: 2-5                         [128, 1024]               [128, 1024]
--
     Linear: 2-6                          [128, 1024]               [128, 512]
524,800
     ReLU: 2-7                            [128, 512]                [128, 512]
--
     Linear: 2-8                          [128, 512]                [128, 100]
51,300
==========================================================================================
==================================
Total params: 964,516
Trainable params: 964,516
Non-trainable params: 0
Total mult-adds (Units.GIGABYTES): 2.01
==========================================================================================
==================================
Input size (MB): 1.57
Forward/backward pass size (MB): 63.54
Params size (MB): 3.86
Estimated Total Size (MB): 68.97
==========================================================================================
==================================
```

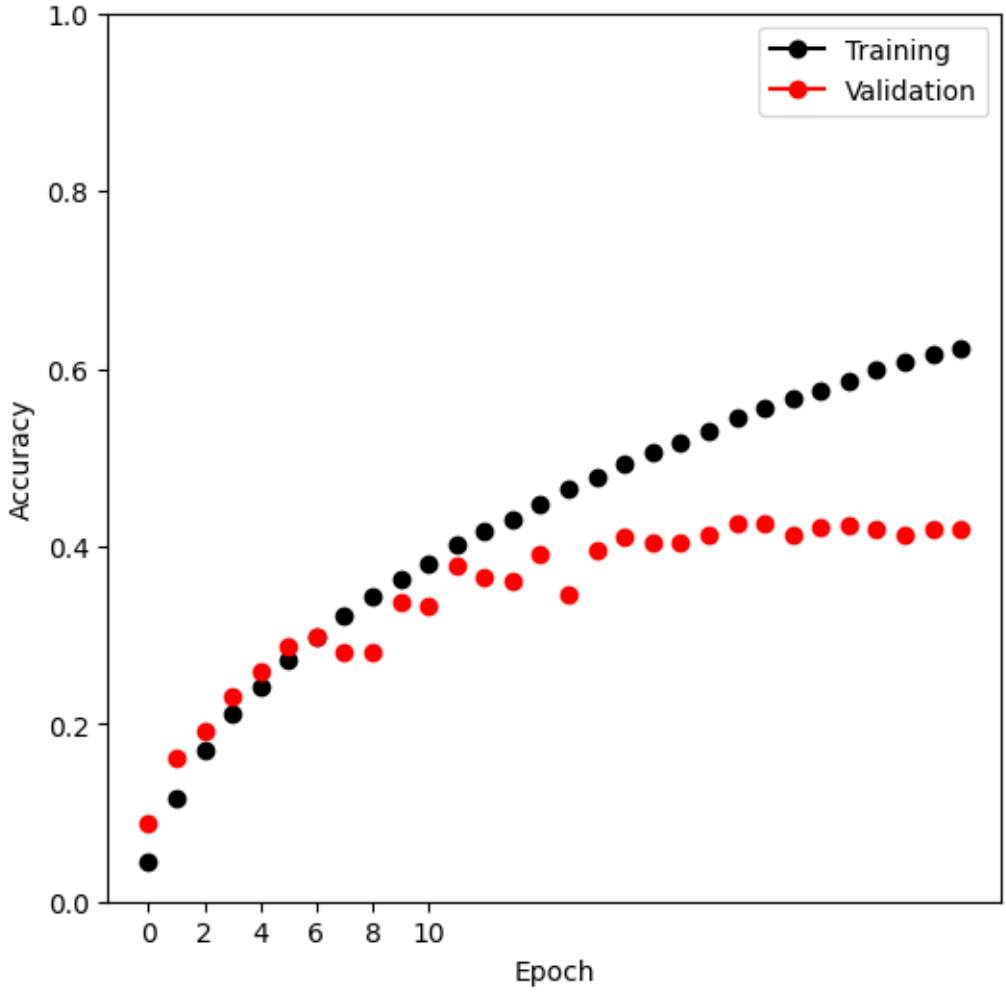
[ ]:
```
cifar_optimizer = RMSprop(cifar_model.parameters(), lr=0.001)
cifar_module = SimpleModule.classification(cifar_model, num_classes=100,␣
↪optimizer=cifar_optimizer)
cifar_logger = CSVLogger('logs', name='CIFAR100')
```

[184]:
```
cifar_trainer = Trainer(deterministic=True, max_epochs=30, logger=cifar_logger,␣
↪enable_progress_bar=False, callbacks=[ErrorTracker()])
```

```
cifar_trainer.fit(cifar_module, datamodule=cifar_dm)
```

```
 Tip: For seamless cloud uploads and versioning, try installing
[litmodels](https://pypi.org/project/litmodels/) to enable LitModelCheckpoint,
which syncs automatically with the Lightning model registry.
GPU available: True (cuda), used: True
TPU available: False, using: 0 TPU cores
LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]

  | Name  | Type             | Params | Mode  | FLOPs
-----------------------------------------------------------
0 | model | CIFARModel       | 964 K  | train | 0
1 | loss  | CrossEntropyLoss | 0      | train | 0
-----------------------------------------------------------
964 K     Trainable params
0         Non-trainable params
964 K     Total params
3.858     Total estimated model params size (MB)
24        Modules in train mode
0         Modules in eval mode
0         Total Flops
`Trainer.fit` stopped: `max_epochs=30` reached.
```

[ ]:
```
log_path = cifar_logger.experiment.metrics_file_path
cifar_results = pd.read_csv(log_path)
fig, ax = subplots(1, 1, figsize=(6, 6))
summary_plot(cifar_results, ax, col='accuracy', ylabel='Accuracy')
ax.set_xticks(np.linspace(0, 10, 6).astype(int))
ax.set_ylabel('Accuracy')
ax.set_ylim([0, 1]);
```



[186]:
```
cifar_trainer.test(cifar_module, datamodule=cifar_dm)
```

```
LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\pytorch_lightning\utilities\_pytree.py:21: `isinstance(treespec,
LeafSpec)` is deprecated, use `isinstance(treespec, TreeSpec) and
treespec.is_leaf()` instead.
```

Test metric	DataLoader 0
test_accuracy	0.41670000553131104
test_loss	2.4101884365081787

[186]: [{'test_loss': 2.4101884365081787, 'test_accuracy': 0.41670000553131104}]

[187]:
```
try:
    for name, metric in cifar_module.metrics.items():
        cifar_module.metrics[name] = metric.to('mps')
    cifar_trainer_mps = Trainer(accelerator='mps', deterministic=True,␣
 ↪enable_progress_bar=False, max_epochs=30)
    cifar_trainer_mps.fit(cifar_module, datamodule=cifar_dm)
    cifar_trainer_mps.test(cifar_module, datamodule=cifar_dm)
except:
    pass
```

[192]:
```
resize = Resize((232,232), antialias=True)
crop = CenterCrop(224)
normalize = Normalize([0.485,0.456,0.406], [0.229,0.224,0.225])
imgfiles = sorted([f for f in glob('./book_images/*')])
imgs = torch.stack([torch.div(crop(resize(read_image(f))), 255) for f in␣
 ↪imgfiles])
imgs = normalize(imgs)
imgs.size()
```

[192]: torch.Size([6, 3, 224, 224])

[193]:
```
resnet_model = resnet50(weights=ResNet50_Weights.DEFAULT)
summary(resnet_model,
        input_data=imgs,
        col_names=['input_size', 'output_size', 'num_params'])
```

```
Downloading: "https://download.pytorch.org/models/resnet50-11ad3fa6.pth" to
C:\Users\archa/.cache\torch\hub\checkpoints\resnet50-11ad3fa6.pth

100%|      | 97.8M/97.8M [00:01<00:00, 53.9MB/s]
```

[193]:
```
==========================================================================================
==================================
Layer (type:depth-idx)                   Input Shape               Output Shape
Param #
==========================================================================================
==================================
ResNet                                   [6, 3, 224, 224]          [6, 1000]
--
 Conv2d: 1-1                             [6, 3, 224, 224]          [6, 64, 112,
112]         9,408
 BatchNorm2d: 1-2                        [6, 64, 112, 112]         [6, 64, 112,
112]         128
 ReLU: 1-3                               [6, 64, 112, 112]         [6, 64, 112,
112]         --
 MaxPool2d: 1-4                          [6, 64, 112, 112]         [6, 64, 56,
```

```
56]           --
 Sequential: 1-5                        [6, 64, 56, 56]           [6, 256, 56,
56]          --
     Bottleneck: 2-1                    [6, 64, 56, 56]           [6, 256, 56,
56]          --
          Conv2d: 3-1                   [6, 64, 56, 56]           [6, 64, 56,
56]           4,096
          BatchNorm2d: 3-2              [6, 64, 56, 56]           [6, 64, 56,
56]           128
          ReLU: 3-3                     [6, 64, 56, 56]           [6, 64, 56,
56]           --
          Conv2d: 3-4                   [6, 64, 56, 56]           [6, 64, 56,
56]           36,864
          BatchNorm2d: 3-5              [6, 64, 56, 56]           [6, 64, 56,
56]           128
          ReLU: 3-6                     [6, 64, 56, 56]           [6, 64, 56,
56]           --
          Conv2d: 3-7                   [6, 64, 56, 56]           [6, 256, 56,
56]          16,384
          BatchNorm2d: 3-8              [6, 256, 56, 56]          [6, 256, 56,
56]          512
          Sequential: 3-9               [6, 64, 56, 56]           [6, 256, 56,
56]          16,896
          ReLU: 3-10                    [6, 256, 56, 56]          [6, 256, 56,
56]          --
     Bottleneck: 2-2                    [6, 256, 56, 56]          [6, 256, 56,
56]          --
          Conv2d: 3-11                  [6, 256, 56, 56]          [6, 64, 56,
56]           16,384
          BatchNorm2d: 3-12             [6, 64, 56, 56]           [6, 64, 56,
56]           128
          ReLU: 3-13                    [6, 64, 56, 56]           [6, 64, 56,
56]           --
          Conv2d: 3-14                  [6, 64, 56, 56]           [6, 64, 56,
56]           36,864
          BatchNorm2d: 3-15             [6, 64, 56, 56]           [6, 64, 56,
56]           128
          ReLU: 3-16                    [6, 64, 56, 56]           [6, 64, 56,
56]           --
          Conv2d: 3-17                  [6, 64, 56, 56]           [6, 256, 56,
56]          16,384
          BatchNorm2d: 3-18             [6, 256, 56, 56]          [6, 256, 56,
56]          512
          ReLU: 3-19                    [6, 256, 56, 56]          [6, 256, 56,
56]          --
     Bottleneck: 2-3                    [6, 256, 56, 56]          [6, 256, 56,
56]          --
```

```
          Conv2d: 3-20                  [6, 256, 56, 56]          [6, 64, 56,
56]           16,384
          BatchNorm2d: 3-21             [6, 64, 56, 56]           [6, 64, 56,
56]           128
          ReLU: 3-22                    [6, 64, 56, 56]           [6, 64, 56,
56]           --
          Conv2d: 3-23                  [6, 64, 56, 56]           [6, 64, 56,
56]           36,864
          BatchNorm2d: 3-24             [6, 64, 56, 56]           [6, 64, 56,
56]           128
          ReLU: 3-25                    [6, 64, 56, 56]           [6, 64, 56,
56]           --
          Conv2d: 3-26                  [6, 64, 56, 56]           [6, 256, 56,
56]          16,384
          BatchNorm2d: 3-27             [6, 256, 56, 56]          [6, 256, 56,
56]          512
          ReLU: 3-28                    [6, 256, 56, 56]          [6, 256, 56,
56]          --
 Sequential: 1-6                         [6, 256, 56, 56]          [6, 512, 28,
28]          --
     Bottleneck: 2-4                     [6, 256, 56, 56]          [6, 512, 28,
28]          --
          Conv2d: 3-29                  [6, 256, 56, 56]          [6, 128, 56,
56]          32,768
          BatchNorm2d: 3-30             [6, 128, 56, 56]          [6, 128, 56,
56]          256
          ReLU: 3-31                    [6, 128, 56, 56]          [6, 128, 56,
56]          --
          Conv2d: 3-32                  [6, 128, 56, 56]          [6, 128, 28,
28]          147,456
          BatchNorm2d: 3-33             [6, 128, 28, 28]          [6, 128, 28,
28]          256
          ReLU: 3-34                    [6, 128, 28, 28]          [6, 128, 28,
28]          --
          Conv2d: 3-35                  [6, 128, 28, 28]          [6, 512, 28,
28]          65,536
          BatchNorm2d: 3-36             [6, 512, 28, 28]          [6, 512, 28,
28]          1,024
          Sequential: 3-37              [6, 256, 56, 56]          [6, 512, 28,
28]          132,096
          ReLU: 3-38                    [6, 512, 28, 28]          [6, 512, 28,
28]          --
     Bottleneck: 2-5                     [6, 512, 28, 28]          [6, 512, 28,
28]          --
          Conv2d: 3-39                  [6, 512, 28, 28]          [6, 128, 28,
28]          65,536
          BatchNorm2d: 3-40             [6, 128, 28, 28]          [6, 128, 28,
```

```
28]          256
         ReLU: 3-41                    [6, 128, 28, 28]          [6, 128, 28,
28]          --
         Conv2d: 3-42                  [6, 128, 28, 28]          [6, 128, 28,
28]          147,456
         BatchNorm2d: 3-43             [6, 128, 28, 28]          [6, 128, 28,
28]          256
         ReLU: 3-44                    [6, 128, 28, 28]          [6, 128, 28,
28]          --
         Conv2d: 3-45                  [6, 128, 28, 28]          [6, 512, 28,
28]          65,536
         BatchNorm2d: 3-46             [6, 512, 28, 28]          [6, 512, 28,
28]          1,024
         ReLU: 3-47                    [6, 512, 28, 28]          [6, 512, 28,
28]          --
     Bottleneck: 2-6                    [6, 512, 28, 28]          [6, 512, 28,
28]          --
         Conv2d: 3-48                  [6, 512, 28, 28]          [6, 128, 28,
28]          65,536
         BatchNorm2d: 3-49             [6, 128, 28, 28]          [6, 128, 28,
28]          256
         ReLU: 3-50                    [6, 128, 28, 28]          [6, 128, 28,
28]          --
         Conv2d: 3-51                  [6, 128, 28, 28]          [6, 128, 28,
28]          147,456
         BatchNorm2d: 3-52             [6, 128, 28, 28]          [6, 128, 28,
28]          256
         ReLU: 3-53                    [6, 128, 28, 28]          [6, 128, 28,
28]          --
         Conv2d: 3-54                  [6, 128, 28, 28]          [6, 512, 28,
28]          65,536
         BatchNorm2d: 3-55             [6, 512, 28, 28]          [6, 512, 28,
28]          1,024
         ReLU: 3-56                    [6, 512, 28, 28]          [6, 512, 28,
28]          --
     Bottleneck: 2-7                    [6, 512, 28, 28]          [6, 512, 28,
28]          --
         Conv2d: 3-57                  [6, 512, 28, 28]          [6, 128, 28,
28]          65,536
         BatchNorm2d: 3-58             [6, 128, 28, 28]          [6, 128, 28,
28]          256
         ReLU: 3-59                    [6, 128, 28, 28]          [6, 128, 28,
28]          --
         Conv2d: 3-60                  [6, 128, 28, 28]          [6, 128, 28,
28]          147,456
         BatchNorm2d: 3-61             [6, 128, 28, 28]          [6, 128, 28,
28]          256
```

```
          ReLU: 3-62                    [6, 128, 28, 28]          [6, 128, 28,
28]          --
          Conv2d: 3-63                  [6, 128, 28, 28]          [6, 512, 28,
28]          65,536
          BatchNorm2d: 3-64             [6, 512, 28, 28]          [6, 512, 28,
28]          1,024
          ReLU: 3-65                    [6, 512, 28, 28]          [6, 512, 28,
28]          --
 Sequential: 1-7                         [6, 512, 28, 28]          [6, 1024, 14,
14]         --
     Bottleneck: 2-8                     [6, 512, 28, 28]          [6, 1024, 14,
14]         --
          Conv2d: 3-66                  [6, 512, 28, 28]          [6, 256, 28,
28]          131,072
          BatchNorm2d: 3-67             [6, 256, 28, 28]          [6, 256, 28,
28]          512
          ReLU: 3-68                    [6, 256, 28, 28]          [6, 256, 28,
28]          --
          Conv2d: 3-69                  [6, 256, 28, 28]          [6, 256, 14,
14]          589,824
          BatchNorm2d: 3-70             [6, 256, 14, 14]          [6, 256, 14,
14]          512
          ReLU: 3-71                    [6, 256, 14, 14]          [6, 256, 14,
14]          --
          Conv2d: 3-72                  [6, 256, 14, 14]          [6, 1024, 14,
14]         262,144
          BatchNorm2d: 3-73             [6, 1024, 14, 14]         [6, 1024, 14,
14]         2,048
          Sequential: 3-74              [6, 512, 28, 28]          [6, 1024, 14,
14]         526,336
          ReLU: 3-75                    [6, 1024, 14, 14]         [6, 1024, 14,
14]         --
     Bottleneck: 2-9                     [6, 1024, 14, 14]         [6, 1024, 14,
14]         --
          Conv2d: 3-76                  [6, 1024, 14, 14]         [6, 256, 14,
14]          262,144
          BatchNorm2d: 3-77             [6, 256, 14, 14]          [6, 256, 14,
14]          512
          ReLU: 3-78                    [6, 256, 14, 14]          [6, 256, 14,
14]          --
          Conv2d: 3-79                  [6, 256, 14, 14]          [6, 256, 14,
14]          589,824
          BatchNorm2d: 3-80             [6, 256, 14, 14]          [6, 256, 14,
14]          512
          ReLU: 3-81                    [6, 256, 14, 14]          [6, 256, 14,
14]          --
          Conv2d: 3-82                  [6, 256, 14, 14]          [6, 1024, 14,
```

```
14]          262,144
          BatchNorm2d: 3-83            [6, 1024, 14, 14]         [6, 1024, 14,
14]          2,048
          ReLU: 3-84                   [6, 1024, 14, 14]         [6, 1024, 14,
14]          --
     Bottleneck: 2-10                  [6, 1024, 14, 14]         [6, 1024, 14,
14]          --
          Conv2d: 3-85                 [6, 1024, 14, 14]         [6, 256, 14,
14]           262,144
          BatchNorm2d: 3-86            [6, 256, 14, 14]          [6, 256, 14,
14]           512
          ReLU: 3-87                   [6, 256, 14, 14]          [6, 256, 14,
14]           --
          Conv2d: 3-88                 [6, 256, 14, 14]          [6, 256, 14,
14]           589,824
          BatchNorm2d: 3-89            [6, 256, 14, 14]          [6, 256, 14,
14]           512
          ReLU: 3-90                   [6, 256, 14, 14]          [6, 256, 14,
14]           --
          Conv2d: 3-91                 [6, 256, 14, 14]          [6, 1024, 14,
14]          262,144
          BatchNorm2d: 3-92            [6, 1024, 14, 14]         [6, 1024, 14,
14]          2,048
          ReLU: 3-93                   [6, 1024, 14, 14]         [6, 1024, 14,
14]          --
     Bottleneck: 2-11                  [6, 1024, 14, 14]         [6, 1024, 14,
14]          --
          Conv2d: 3-94                 [6, 1024, 14, 14]         [6, 256, 14,
14]           262,144
          BatchNorm2d: 3-95            [6, 256, 14, 14]          [6, 256, 14,
14]           512
          ReLU: 3-96                   [6, 256, 14, 14]          [6, 256, 14,
14]           --
          Conv2d: 3-97                 [6, 256, 14, 14]          [6, 256, 14,
14]           589,824
          BatchNorm2d: 3-98            [6, 256, 14, 14]          [6, 256, 14,
14]           512
          ReLU: 3-99                   [6, 256, 14, 14]          [6, 256, 14,
14]           --
          Conv2d: 3-100                [6, 256, 14, 14]          [6, 1024, 14,
14]          262,144
          BatchNorm2d: 3-101           [6, 1024, 14, 14]         [6, 1024, 14,
14]          2,048
          ReLU: 3-102                  [6, 1024, 14, 14]         [6, 1024, 14,
14]          --
     Bottleneck: 2-12                  [6, 1024, 14, 14]         [6, 1024, 14,
14]          --
```

```
          Conv2d: 3-103                [6, 1024, 14, 14]         [6, 256, 14,
14]          262,144
          BatchNorm2d: 3-104           [6, 256, 14, 14]          [6, 256, 14,
14]          512
          ReLU: 3-105                  [6, 256, 14, 14]          [6, 256, 14,
14]          --
          Conv2d: 3-106                [6, 256, 14, 14]          [6, 256, 14,
14]          589,824
          BatchNorm2d: 3-107           [6, 256, 14, 14]          [6, 256, 14,
14]          512
          ReLU: 3-108                  [6, 256, 14, 14]          [6, 256, 14,
14]          --
          Conv2d: 3-109                [6, 256, 14, 14]          [6, 1024, 14,
14]         262,144
          BatchNorm2d: 3-110           [6, 1024, 14, 14]         [6, 1024, 14,
14]         2,048
          ReLU: 3-111                  [6, 1024, 14, 14]         [6, 1024, 14,
14]         --
     Bottleneck: 2-13                   [6, 1024, 14, 14]         [6, 1024, 14,
14]         --
          Conv2d: 3-112                [6, 1024, 14, 14]         [6, 256, 14,
14]          262,144
          BatchNorm2d: 3-113           [6, 256, 14, 14]          [6, 256, 14,
14]          512
          ReLU: 3-114                  [6, 256, 14, 14]          [6, 256, 14,
14]          --
          Conv2d: 3-115                [6, 256, 14, 14]          [6, 256, 14,
14]          589,824
          BatchNorm2d: 3-116           [6, 256, 14, 14]          [6, 256, 14,
14]          512
          ReLU: 3-117                  [6, 256, 14, 14]          [6, 256, 14,
14]          --
          Conv2d: 3-118                [6, 256, 14, 14]          [6, 1024, 14,
14]         262,144
          BatchNorm2d: 3-119           [6, 1024, 14, 14]         [6, 1024, 14,
14]         2,048
          ReLU: 3-120                  [6, 1024, 14, 14]         [6, 1024, 14,
14]         --
 Sequential: 1-8                       [6, 1024, 14, 14]         [6, 2048, 7,
7]           --
     Bottleneck: 2-14                   [6, 1024, 14, 14]         [6, 2048, 7,
7]           --
          Conv2d: 3-121                [6, 1024, 14, 14]         [6, 512, 14,
14]          524,288
          BatchNorm2d: 3-122           [6, 512, 14, 14]          [6, 512, 14,
14]          1,024
          ReLU: 3-123                  [6, 512, 14, 14]          [6, 512, 14,
```

```
14]          --
          Conv2d: 3-124                [6, 512, 14, 14]          [6, 512, 7,
7]            2,359,296
          BatchNorm2d: 3-125           [6, 512, 7, 7]            [6, 512, 7,
7]            1,024
          ReLU: 3-126                  [6, 512, 7, 7]            [6, 512, 7,
7]            --
          Conv2d: 3-127                [6, 512, 7, 7]            [6, 2048, 7,
7]           1,048,576
          BatchNorm2d: 3-128           [6, 2048, 7, 7]           [6, 2048, 7,
7]           4,096
          Sequential: 3-129            [6, 1024, 14, 14]         [6, 2048, 7,
7]           2,101,248
          ReLU: 3-130                  [6, 2048, 7, 7]           [6, 2048, 7,
7]           --
      Bottleneck: 2-15                 [6, 2048, 7, 7]           [6, 2048, 7,
7]           --
          Conv2d: 3-131                [6, 2048, 7, 7]           [6, 512, 7,
7]            1,048,576
          BatchNorm2d: 3-132           [6, 512, 7, 7]            [6, 512, 7,
7]            1,024
          ReLU: 3-133                  [6, 512, 7, 7]            [6, 512, 7,
7]            --
          Conv2d: 3-134                [6, 512, 7, 7]            [6, 512, 7,
7]            2,359,296
          BatchNorm2d: 3-135           [6, 512, 7, 7]            [6, 512, 7,
7]            1,024
          ReLU: 3-136                  [6, 512, 7, 7]            [6, 512, 7,
7]            --
          Conv2d: 3-137                [6, 512, 7, 7]            [6, 2048, 7,
7]           1,048,576
          BatchNorm2d: 3-138           [6, 2048, 7, 7]           [6, 2048, 7,
7]           4,096
          ReLU: 3-139                  [6, 2048, 7, 7]           [6, 2048, 7,
7]           --
      Bottleneck: 2-16                 [6, 2048, 7, 7]           [6, 2048, 7,
7]           --
          Conv2d: 3-140                [6, 2048, 7, 7]           [6, 512, 7,
7]            1,048,576
          BatchNorm2d: 3-141           [6, 512, 7, 7]            [6, 512, 7,
7]            1,024
          ReLU: 3-142                  [6, 512, 7, 7]            [6, 512, 7,
7]            --
          Conv2d: 3-143                [6, 512, 7, 7]            [6, 512, 7,
7]            2,359,296
          BatchNorm2d: 3-144           [6, 512, 7, 7]            [6, 512, 7,
7]            1,024
```

```
          ReLU: 3-145                    [6, 512, 7, 7]            [6, 512, 7,
7]            --
          Conv2d: 3-146                  [6, 512, 7, 7]            [6, 2048, 7,
7]           1,048,576
          BatchNorm2d: 3-147             [6, 2048, 7, 7]           [6, 2048, 7,
7]           4,096
          ReLU: 3-148                    [6, 2048, 7, 7]           [6, 2048, 7,
7]           --
 AdaptiveAvgPool2d: 1-9                   [6, 2048, 7, 7]           [6, 2048, 1,
1]           --
 Linear: 1-10                             [6, 2048]                 [6, 1000]
2,049,000
==========================================================================
===================================
Total params: 25,557,032
Trainable params: 25,557,032
Non-trainable params: 0
Total mult-adds (Units.GIGABYTES): 24.54
==========================================================================
===================================
Input size (MB): 3.61
Forward/backward pass size (MB): 1066.99
Params size (MB): 102.23
Estimated Total Size (MB): 1172.83
==========================================================================
===================================
```

[194]:
```
resnet_model.eval()
```

[194]:
```
ResNet(
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3),
bias=False)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (relu): ReLU(inplace=True)
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1,
ceil_mode=False)
  (layer1): Sequential(
    (0): Bottleneck(
      (conv1): Conv2d(64, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
```

```
      (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
      (downsample): Sequential(
        (0): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      )
    )
    (1): Bottleneck(
      (conv1): Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
    (2): Bottleneck(
      (conv1): Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
  )
  (layer2): Sequential(
    (0): Bottleneck(
      (conv1): Conv2d(256, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
```

```
track_running_stats=True)
      (relu): ReLU(inplace=True)
      (downsample): Sequential(
        (0): Conv2d(256, 512, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      )
    )
    (1): Bottleneck(
      (conv1): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
    (2): Bottleneck(
      (conv1): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
    (3): Bottleneck(
      (conv1): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
```

```
  )
  (layer3): Sequential(
    (0): Bottleneck(
      (conv1): Conv2d(512, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
      (downsample): Sequential(
        (0): Conv2d(512, 1024, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      )
    )
    (1): Bottleneck(
      (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
    (2): Bottleneck(
      (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
    (3): Bottleneck(
```

```
      (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
    (4): Bottleneck(
      (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
    (5): Bottleneck(
      (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
  )
  (layer4): Sequential(
    (0): Bottleneck(
      (conv1): Conv2d(1024, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(2, 2), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
```

```
track_running_stats=True)
      (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
      (downsample): Sequential(
        (0): Conv2d(1024, 2048, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      )
    )
    (1): Bottleneck(
      (conv1): Conv2d(2048, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
    (2): Bottleneck(
      (conv1): Conv2d(2048, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
  )
  (avgpool): AdaptiveAvgPool2d(output_size=(1, 1))
  (fc): Linear(in_features=2048, out_features=1000, bias=True)
)
```

[195]:
```
img_preds = resnet_model(imgs)
```

[196]:
```
img_probs = np.exp(np.asarray(img_preds.detach()))
img_probs /= img_probs.sum(1)[:,None]
```

[197]:
```
labs = json.load(open('./imagenet_class_index.json'))
class_labels = pd.DataFrame([(int(k), v[1]) for k, v in
                            labs.items()],
                            columns=['idx', 'label'])
class_labels = class_labels.set_index('idx')
class_labels = class_labels.sort_index()
```

[198]:
```
for i, imgfile in enumerate(imgfiles):
    img_df = class_labels.copy()
    img_df['prob'] = img_probs[i]
    img_df = img_df.sort_values(by='prob', ascending=False)[:3]
    print(f'Image: {imgfile}')
    print(img_df.reset_index().drop(columns=['idx']))
```

```
Image: ./book_images\Cape_Weaver.jpg
      label      prob
0   jacamar  0.297500
1     macaw  0.068107
2  lorikeet  0.051104
Image: ./book_images\Flamingo.jpg
            label      prob
0        flamingo  0.609514
1       spoonbill  0.013586
2  American_egret  0.002132
Image: ./book_images\Hawk_Fountain.jpg
            label      prob
0            kite  0.184716
1           robin  0.084018
2  great_grey_owl  0.061267
Image: ./book_images\Hawk_cropped.jpg
            label      prob
0            kite  0.453834
1  great_grey_owl  0.015914
2             jay  0.012210
Image: ./book_images\Lhasa_Apso.jpg
             label      prob
0            Lhasa  0.260317
1         Shih-Tzu  0.097196
2  Tibetan_terrier  0.032820
Image: ./book_images\Sleeping_Cat.jpg
         label      prob
0  Persian_cat  0.163070
1        tabby  0.074143
2    tiger_cat  0.042578
```

[199]:
```
del(cifar_test,
    cifar_train,
    cifar_dm,
```

```
    cifar_module,
    cifar_logger,
    cifar_optimizer,
    cifar_trainer)
```

[200]:
```
(imdb_seq_train,
 imdb_seq_test) = load_sequential(root='data/IMDB')
padded_sample = np.asarray(imdb_seq_train.tensors[0][0])
sample_review = padded_sample[padded_sample > 0][:12]
sample_review[:12]
```

```
Retrieving "IMDB_S_train.tensor.gz" from
"http://imdb.jtaylor.su.domains/jtaylor/data/".
Retrieving "IMDB_S_test.tensor.gz" from
"http://imdb.jtaylor.su.domains/jtaylor/data/".
Retrieving "IMDB_Y_test.npy" from
"http://imdb.jtaylor.su.domains/jtaylor/data/".
Retrieving "IMDB_Y_train.npy" from
"http://imdb.jtaylor.su.domains/jtaylor/data/".
```

[200]:
```
array([   1,   14,   22,   16,   43,  530,  973, 1622, 1385,   65,  458,
       4468], dtype=int32)
```

[201]:
```
lookup = load_lookup(root='data/IMDB')
' '.join(lookup[i] for i in sample_review)
```

```
Retrieving "IMDB_word_index.pkl" from
"http://imdb.jtaylor.su.domains/jtaylor/data/".
```

[201]:
```
"<START> this film was just brilliant casting location scenery story direction
 everyone's"
```

[202]:
```
max_num_workers=10
(imdb_train,
 imdb_test) = load_tensor(root='data/IMDB')
imdb_dm = SimpleDataModule(imdb_train, imdb_test, validation=2000,␣
 ↪num_workers=min(6, max_num_workers), batch_size=512)
```

```
Retrieving "IMDB_X_test.tensor.gz" from
"http://imdb.jtaylor.su.domains/jtaylor/data/".
Retrieving "IMDB_X_train.tensor.gz" from
"http://imdb.jtaylor.su.domains/jtaylor/data/".
```

[203]:
```
class IMDBModel(nn.Module):

    def __init__(self, input_size):
        super(IMDBModel, self).__init__()
        self.dense1 = nn.Linear(input_size, 16)
        self.activation = nn.ReLU()
```

```
        self.dense2 = nn.Linear(16, 16)
        self.output = nn.Linear(16, 1)

    def forward(self, x):
        val = x
        for _map in [self.dense1,
                     self.activation,
                     self.dense2,
                     self.activation,
                     self.output]:
            val = _map(val)
        return torch.flatten(val)
```

[ ]:
```
imdb_model = IMDBModel(imdb_test.tensors[0].size()[1])
summary(imdb_model,
        input_size=imdb_test.tensors[0].size(),
        col_names=['input_size', 'output_size', 'num_params'])
```

[ ]:
```
==================================================================================
===================================
Layer (type:depth-idx)                   Input Shape               Output Shape
Param #
==================================================================================
===================================
IMDBModel                                [25000, 10003]            [25000]
--
 Linear: 1-1                             [25000, 10003]            [25000, 16]
160,064
 ReLU: 1-2                               [25000, 16]               [25000, 16]
--
 Linear: 1-3                             [25000, 16]               [25000, 16]
272
 ReLU: 1-4                               [25000, 16]               [25000, 16]
--
 Linear: 1-5                             [25000, 16]               [25000, 1]
17
==================================================================================
===================================
Total params: 160,353
Trainable params: 160,353
Non-trainable params: 0
Total mult-adds (Units.GIGABYTES): 4.01
==================================================================================
===================================
Input size (MB): 1000.30
Forward/backward pass size (MB): 6.60
Params size (MB): 0.64
```

```
Estimated Total Size (MB): 1007.54
================================================================================
==================================
```

[ ]:
```
imdb_optimizer = RMSprop(imdb_model.parameters(), lr=0.001)
imdb_module = SimpleModule.binary_classification(imdb_model,␣
↪optimizer=imdb_optimizer)
```

[206]:
```
imdb_logger = CSVLogger('logs', name='IMDB')
imdb_trainer = Trainer(deterministic=True, max_epochs=30, logger=imdb_logger,␣
↪enable_progress_bar=False, callbacks=[ErrorTracker()])
imdb_trainer.fit(imdb_module, datamodule=imdb_dm)
```

```
 Tip: For seamless cloud uploads and versioning, try installing
[litmodels](https://pypi.org/project/litmodels/) to enable LitModelCheckpoint,
which syncs automatically with the Lightning model registry.
GPU available: True (cuda), used: True
TPU available: False, using: 0 TPU cores
LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]

  | Name  | Type              | Params | Mode  | FLOPs
-----------------------------------------------------------
0 | model | IMDBModel         | 160 K  | train | 0
1 | loss  | BCEWithLogitsLoss | 0      | train | 0
-----------------------------------------------------------
160 K     Trainable params
0         Non-trainable params
160 K     Total params
0.641     Total estimated model params size (MB)
6         Modules in train mode
0         Modules in eval mode
0         Total Flops
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\pytorch_lightning\utilities\_pytree.py:21: `isinstance(treespec,
LeafSpec)` is deprecated, use `isinstance(treespec, TreeSpec) and
treespec.is_leaf()` instead.
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\pytorch_lightning\loops\fit_loop.py:317: The number of training batches
(45) is smaller than the logging interval Trainer(log_every_n_steps=50). Set a
lower value for log_every_n_steps if you want to see logs for the training
epoch.
`Trainer.fit` stopped: `max_epochs=30` reached.
```

[207]:
```
test_results = imdb_trainer.test(imdb_module, datamodule=imdb_dm)
test_results
```

```
LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]
```

```
       Test metric             DataLoader 0

      test_accuracy         0.8505200147628784
        test_loss            1.130419135093689
```

[207]: [{'test_loss': 1.130419135093689, 'test_accuracy': 0.8505200147628784}]

[208]:
```
((X_train, Y_train), (X_valid, Y_valid), (X_test, Y_test)) =␣
↪load_sparse(validation=2000, random_state=0, root='data/IMDB')
```

```
Retrieving "IMDB_X_test.npz" from
"http://imdb.jtaylor.su.domains/jtaylor/data/".
Retrieving "IMDB_X_train.npz" from
"http://imdb.jtaylor.su.domains/jtaylor/data/".
```

[ ]:
```
lam_max = np.abs(X_train.T * (Y_train - Y_train.mean())).max()
lam_val = lam_max * np.exp(np.linspace(np.log(1), np.log(1e-4), 50))
```

[210]:
```
logit = LogisticRegression(penalty='l1', C=1/lam_max, solver='liblinear',␣
↪warm_start=True, fit_intercept=True)
```

[211]:
```
coefs = []
intercepts = []

for l in lam_val:
    logit.C = 1/l
    logit.fit(X_train, Y_train)
    coefs.append(logit.coef_.copy())
    intercepts.append(logit.intercept_)
```

```
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
```

```
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
```

```
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
```

```
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
```

```
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
```

```
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
```

```
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
```

```
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
```

```
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
```

```
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
```

```
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
```

```
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
```

```
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
```

```
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
  warnings.warn(
```

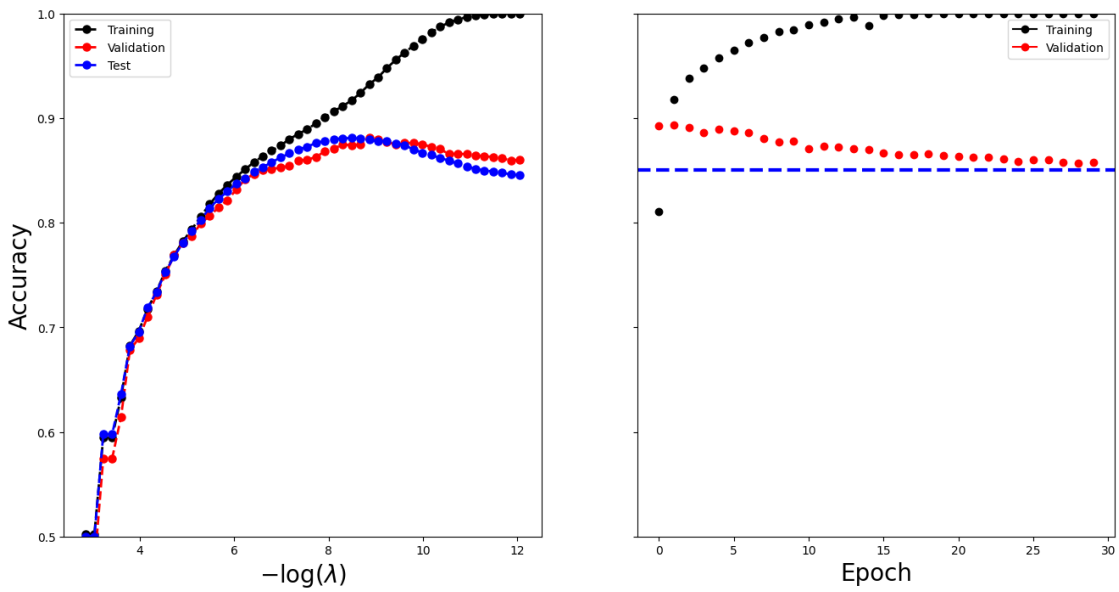
[212]:
```
coefs = np.squeeze(coefs)
intercepts = np.squeeze(intercepts)
```

[213]:
```
%%capture
fig, axes = subplots(1, 2, figsize=(16, 8), sharey=True)
for ((X_, Y_),
     data_,
     color) in zip([(X_train, Y_train),
                    (X_valid, Y_valid),
                    (X_test, Y_test)],
                    ['Training', 'Validation', 'Test'],
                    ['black', 'red', 'blue']):
    linpred_ = X_ * coefs.T + intercepts[None,:]
    label_ = np.array(linpred_ > 0)
    accuracy_ = np.array([np.mean(Y_ == l) for l in label_.T])
    axes[0].plot(-np.log(lam_val / X_train.shape[0]),
                 accuracy_,
                 '.--',
                 color=color,
                 markersize=13,
                 linewidth=2,
                 label=data_)
axes[0].legend()
axes[0].set_xlabel(r'$-\log(\lambda)$', fontsize=20)
axes[0].set_ylabel('Accuracy', fontsize=20)
```

[214]:
```
imdb_results = pd.read_csv(imdb_logger.experiment.metrics_file_path)
summary_plot(imdb_results,
             axes[1],
             col='accuracy',
             ylabel='Accuracy')
axes[1].set_xticks(np.linspace(0, 30, 7).astype(int))
axes[1].set_ylabel('Accuracy', fontsize=20)
axes[1].set_xlabel('Epoch', fontsize=20)
axes[1].set_ylim([0.5, 1]);
axes[1].axhline(test_results[0]['test_accuracy'],
                color='blue',
```

```
                 linestyle='--',
                 linewidth=3)
fig
```

[214]:



[215]:
```
del(imdb_model,
    imdb_trainer,
    imdb_logger,
    imdb_dm,
    imdb_train,
    imdb_test)
```

[216]:
```
imdb_seq_dm = SimpleDataModule(imdb_seq_train, imdb_seq_test, validation=2000,␣
↪batch_size=300, num_workers=min(6, max_num_workers))
```

[217]:
```
class LSTMModel(nn.Module):
    def __init__(self, input_size):
        super(LSTMModel, self).__init__()
        self.embedding = nn.Embedding(input_size, 32)
        self.lstm = nn.LSTM(input_size=32,
                            hidden_size=32,
                            batch_first=True)
        self.dense = nn.Linear(32, 1)
    def forward(self, x):
        val, (h_n, c_n) = self.lstm(self.embedding(x))
        return torch.flatten(self.dense(val[:,-1]))
```

[218]:
```
lstm_model = LSTMModel(X_test.shape[-1])
summary(lstm_model,
        input_data=imdb_seq_train.tensors[0][:10],
        col_names=['input_size',
                   'output_size',
                   'num_params'])
```

[218]:
```
==========================================================================================
==================================
Layer (type:depth-idx)                   Input Shape               Output Shape
Param #
==========================================================================================
==================================
LSTMModel                                [10, 500]                 [10]
--
 Embedding: 1-1                          [10, 500]                 [10, 500, 32]
320,096
 LSTM: 1-2                               [10, 500, 32]             [10, 500, 32]
8,448
 Linear: 1-3                             [10, 32]                  [10, 1]
33
==========================================================================================
==================================
Total params: 328,577
Trainable params: 328,577
Non-trainable params: 0
Total mult-adds (Units.MEGABYTES): 45.44
==========================================================================================
==================================
Input size (MB): 50.00
Forward/backward pass size (MB): 2.56
Params size (MB): 1.31
Estimated Total Size (MB): 53.87
==========================================================================================
==================================
```

[219]:
```
lstm_module = SimpleModule.binary_classification(lstm_model)
lstm_logger = CSVLogger('logs', name='IMDB_LSTM')
```

[220]:
```
lstm_trainer = Trainer(deterministic=True, max_epochs=20, logger=lstm_logger,␣
 ↪enable_progress_bar=False, callbacks=[ErrorTracker()])
lstm_trainer.fit(lstm_module, datamodule=imdb_seq_dm)
```

```
 Tip: For seamless cloud uploads and versioning, try installing
[litmodels](https://pypi.org/project/litmodels/) to enable LitModelCheckpoint,
which syncs automatically with the Lightning model registry.
GPU available: True (cuda), used: True
TPU available: False, using: 0 TPU cores
```

```
LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]

  | Name  | Type              | Params | Mode  | FLOPs
-----------------------------------------------------------
0 | model | LSTMModel         | 328 K  | train | 0
1 | loss  | BCEWithLogitsLoss | 0      | train | 0
-----------------------------------------------------------
328 K     Trainable params
0         Non-trainable params
328 K     Total params
1.314     Total estimated model params size (MB)
5         Modules in train mode
0         Modules in eval mode
0         Total Flops
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\pytorch_lightning\utilities\_pytree.py:21: `isinstance(treespec,
LeafSpec)` is deprecated, use `isinstance(treespec, TreeSpec) and
treespec.is_leaf()` instead.
`Trainer.fit` stopped: `max_epochs=20` reached.
```

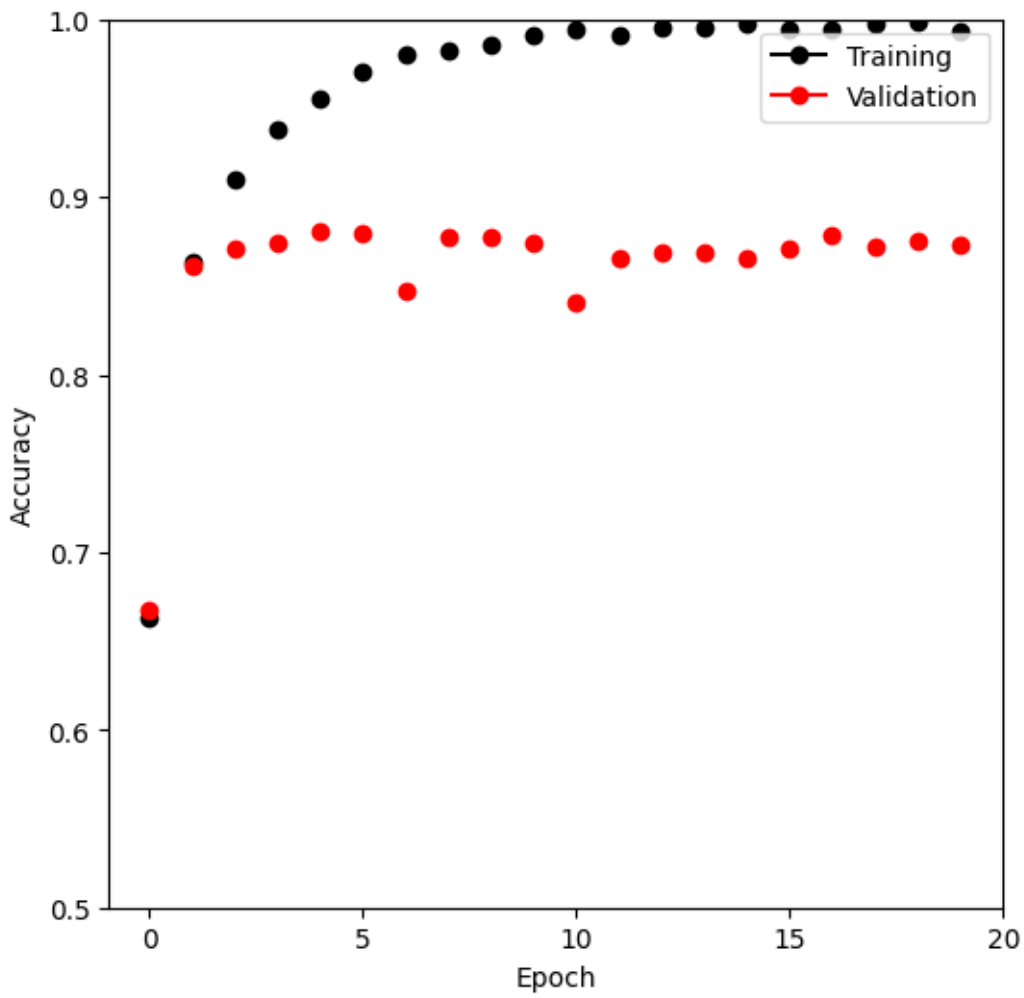
[221]:
```
lstm_trainer.test(lstm_module, datamodule=imdb_seq_dm)
```

```
LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]


       Test metric             DataLoader 0


      test_accuracy         0.8616799712181091
        test_loss            0.748612105846405
```

[221]:
```
[{'test_loss': 0.748612105846405, 'test_accuracy': 0.8616799712181091}]
```

[222]:
```
lstm_results = pd.read_csv(lstm_logger.experiment.metrics_file_path)
fig, ax = subplots(1, 1, figsize=(6, 6))
summary_plot(lstm_results,
             ax,
             col='accuracy',
             ylabel='Accuracy')
ax.set_xticks(np.linspace(0, 20, 5).astype(int))
ax.set_ylabel('Accuracy')
ax.set_ylim([0.5, 1])
```

[222]:
```
(0.5, 1.0)
```



[223]:
```
del(lstm_model,
    lstm_trainer,
    lstm_logger,
    imdb_seq_dm,
    imdb_seq_train,
    imdb_seq_test)
```

[224]:
```
NYSE = load_data('NYSE')
cols = ['DJ_return', 'log_volume', 'log_volatility']
X = pd.DataFrame(StandardScaler(
                     with_mean=True,
                     with_std=True).fit_transform(NYSE[cols]),
                 columns=NYSE[cols].columns,
                 index=NYSE.index)
```

[225]:
```
for lag in range(1, 6):
    for col in cols:
        newcol = np.zeros(X.shape[0]) * np.nan
        newcol[lag:] = X[col].values[:-lag]
        X.insert(len(X.columns), "{0}_{1}".format(col, lag), newcol)
X.insert(len(X.columns), 'train', NYSE['train'])
X = X.dropna()
```

[226]:
```
Y, train = X['log_volume'], X['train']
X = X.drop(columns=['train'] + cols)
X.columns
```

[226]:
```
Index(['DJ_return_1', 'log_volume_1', 'log_volatility_1', 'DJ_return_2',
       'log_volume_2', 'log_volatility_2', 'DJ_return_3', 'log_volume_3',
       'log_volatility_3', 'DJ_return_4', 'log_volume_4', 'log_volatility_4',
       'DJ_return_5', 'log_volume_5', 'log_volatility_5'],
      dtype='str')
```

[227]:
```
M = LinearRegression()
M.fit(X[train], Y[train])
M.score(X[~train], Y[~train])
```

[227]:
```
0.41289129385625223
```

[ ]:
```
X_day = pd.concat([X, pd.get_dummies(NYSE['day_of_week'])], axis=1).dropna()
```

[229]:
```
M.fit(X_day[train], Y[train])
M.score(X_day[~train], Y[~train])
```

[229]:
```
0.4595563133053274
```

[230]:
```
ordered_cols = []
for lag in range(5,0,-1):
    for col in cols:
        ordered_cols.append('{0}_{1}'.format(col, lag))
X = X.reindex(columns=ordered_cols)
X.columns
```

[230]:
```
Index(['DJ_return_5', 'log_volume_5', 'log_volatility_5', 'DJ_return_4',
       'log_volume_4', 'log_volatility_4', 'DJ_return_3', 'log_volume_3',
       'log_volatility_3', 'DJ_return_2', 'log_volume_2', 'log_volatility_2',
       'DJ_return_1', 'log_volume_1', 'log_volatility_1'],
      dtype='str')
```

[231]:
```
X_rnn = X.to_numpy().reshape((-1,5,3))
X_rnn.shape
```

[231]:
```
(6046, 5, 3)
```

[232]:
```
class NYSEModel(nn.Module):
    def __init__(self):
        super(NYSEModel, self).__init__()
        self.rnn = nn.RNN(3,
                          12,
                          batch_first=True)
        self.dense = nn.Linear(12, 1)
        self.dropout = nn.Dropout(0.1)
    def forward(self, x):
        val, h_n = self.rnn(x)
        val = self.dense(self.dropout(val[:,-1]))
        return torch.flatten(val)
nyse_model = NYSEModel()
```

[235]:
```
datasets = []
for mask in [train, ~train]:
    X_rnn_t = torch.tensor(X_rnn[mask].astype(np.float32))
    Y_t = torch.tensor(Y[mask].astype(np.float32).to_numpy())
    datasets.append(TensorDataset(X_rnn_t, Y_t))
nyse_train, nyse_test = datasets
```

[236]:
```
summary(nyse_model,
        input_data=X_rnn_t,
        col_names=['input_size',
                   'output_size',
                   'num_params'])
```

[236]:
```
==========================================================================
===================================
Layer (type:depth-idx)                   Input Shape               Output Shape
Param #
==========================================================================
===================================
NYSEModel                                [1770, 5, 3]              [1770]
--
 RNN: 1-1                                [1770, 5, 3]              [1770, 5, 12]
204
 Dropout: 1-2                            [1770, 12]                [1770, 12]
--
 Linear: 1-3                             [1770, 12]                [1770, 1]
13
==========================================================================
===================================
Total params: 217
Trainable params: 217
Non-trainable params: 0
Total mult-adds (Units.MEGABYTES): 1.83
```

```
================================================================================
==================================
Input size (MB): 0.11
Forward/backward pass size (MB): 0.86
Params size (MB): 0.00
Estimated Total Size (MB): 0.97
================================================================================
==================================
```

[237]:
```
nyse_dm = SimpleDataModule(nyse_train, nyse_test, num_workers=min(4,␣
↪max_num_workers), validation=nyse_test, batch_size=64)
```

[238]:
```
for idx, (x, y) in enumerate(nyse_dm.train_dataloader()):
    out = nyse_model(x)
    print(y.size(), out.size())
    if idx >= 2:
        break
```

```
torch.Size([64]) torch.Size([64])
torch.Size([64]) torch.Size([64])
torch.Size([64]) torch.Size([64])
```

[ ]:
```
nyse_optimizer = RMSprop(nyse_model.parameters(), lr=0.001)
nyse_module = SimpleModule.regression(nyse_model, optimizer=nyse_optimizer,␣
↪metrics={'r2':R2Score()})
```

[240]:
```
nyse_trainer = Trainer(deterministic=True, max_epochs=200,␣
↪enable_progress_bar=False, callbacks=[ErrorTracker()])
nyse_trainer.fit(nyse_module, datamodule=nyse_dm)
nyse_trainer.test(nyse_module, datamodule=nyse_dm)
```

```
 Tip: For seamless cloud uploads and versioning, try installing
[litmodels](https://pypi.org/project/litmodels/) to enable LitModelCheckpoint,
which syncs automatically with the Lightning model registry.
GPU available: True (cuda), used: True
TPU available: False, using: 0 TPU cores
LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]

  | Name  | Type      | Params | Mode  | FLOPs
-----------------------------------------------------
0 | model | NYSEModel | 217    | train | 0
1 | loss  | MSELoss   | 0      | train | 0
-----------------------------------------------------
217       Trainable params
0         Non-trainable params
217       Total params
0.001     Total estimated model params size (MB)
5         Modules in train mode
```

```
0         Modules in eval mode
0         Total Flops
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\pytorch_lightning\utilities\_pytree.py:21: `isinstance(treespec,
LeafSpec)` is deprecated, use `isinstance(treespec, TreeSpec) and
treespec.is_leaf()` instead.
`Trainer.fit` stopped: `max_epochs=200` reached.
LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]


       Test metric             DataLoader 0


        test_loss           0.6248728632926941
         test_r2            0.4069651961326599
```

[240]:
```
[{'test_loss': 0.6248728632926941, 'test_r2': 0.4069651961326599}]
```

[241]:
```
datasets = []
for mask in [train, ~train]:
    X_day_t = torch.tensor(
                    np.asarray(X_day[mask]).astype(np.float32))
    Y_t = torch.tensor(np.asarray(Y[mask]).astype(np.float32))
    datasets.append(TensorDataset(X_day_t, Y_t))
day_train, day_test = datasets
```

[242]:
```
day_dm = SimpleDataModule(day_train, day_test, num_workers=min(4,␣
↪max_num_workers), validation=day_test, batch_size=64)
```

[243]:
```
class NonLinearARModel(nn.Module):
    def __init__(self):
        super(NonLinearARModel, self).__init__()
        self._forward = nn.Sequential(nn.Flatten(),
                                      nn.Linear(20, 32),
                                      nn.ReLU(),
                                      nn.Dropout(0.5),
                                      nn.Linear(32, 1))
    def forward(self, x):
        return torch.flatten(self._forward(x))
```

[244]:
```
nl_model = NonLinearARModel()
nl_optimizer = RMSprop(nl_model.parameters(), lr=0.001)
nl_module = SimpleModule.regression(nl_model, optimizer=nl_optimizer,␣
↪metrics={'r2':R2Score()})
```

[245]:
```
nl_trainer = Trainer(deterministic=True, max_epochs=20,␣
↪enable_progress_bar=False, callbacks=[ErrorTracker()])
nl_trainer.fit(nl_module, datamodule=day_dm)
nl_trainer.test(nl_module, datamodule=day_dm)
```

```
 Tip: For seamless cloud uploads and versioning, try installing
[litmodels](https://pypi.org/project/litmodels/) to enable LitModelCheckpoint,
which syncs automatically with the Lightning model registry.
GPU available: True (cuda), used: True
TPU available: False, using: 0 TPU cores
LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]

  | Name  | Type             | Params | Mode  | FLOPs
-----------------------------------------------------------
0 | model | NonLinearARModel | 705    | train | 0
1 | loss  | MSELoss          | 0      | train | 0
-----------------------------------------------------------
705       Trainable params
0         Non-trainable params
705       Total params
0.003     Total estimated model params size (MB)
8         Modules in train mode
0         Modules in eval mode
0         Total Flops
`Trainer.fit` stopped: `max_epochs=20` reached.
LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]


       Test metric             DataLoader 0


        test_loss           0.5648060441017151
         test_r2            0.46397149562835693
```

[245]:
```
[{'test_loss': 0.5648060441017151, 'test_r2': 0.46397149562835693}]
```

[250]:
```
import numpy as np
%matplotlib inline
import matplotlib.pyplot as plt
from PIL import Image
import torch
from torchvision import datasets, models, transforms
import torch.nn as nn
from torch.nn import functional as F
import torch.optim as optim
```

```
import torchvision

input_path = "./archive/"
```

[251]:
```
normalize = transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224,␣
↪0.225])

data_transforms = {
    'train':
    transforms.Compose([
        transforms.Resize((224,224)),
        transforms.RandomAffine(0, shear=10, scale=(0.8,1.2)),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        normalize
    ]),
    'validation':
    transforms.Compose([
        transforms.Resize((224,224)),
        transforms.ToTensor(),
        normalize
    ]),
}

image_datasets = {
    'train': datasets.ImageFolder(input_path + 'training_set/training_set',␣
↪data_transforms['train']),
    'validation': datasets.ImageFolder(input_path + 'test_set/test_set',␣
↪data_transforms['validation'])
}

dataloaders = {
    'train': torch.utils.data.DataLoader(image_datasets['train'],␣
↪batch_size=32, shuffle=True, num_workers=0),  # for Kaggle
    'validation': torch.utils.data.DataLoader(image_datasets['validation'],␣
↪batch_size=32, shuffle=False, num_workers=0)  # for Kaggle
}
```

[252]:
```
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
device
```

[252]:
```
device(type='cuda', index=0)
```

[253]:
```
model = models.resnet50(pretrained=True).to(device)

for param in model.parameters():
    param.requires_grad = False
```

```
model.fc = nn.Sequential(
               nn.Linear(2048, 128),
               nn.ReLU(inplace=True),
               nn.Linear(128, 2)).to(device)
```

```
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\torchvision\models\_utils.py:208: UserWarning: The parameter
'pretrained' is deprecated since 0.13 and may be removed in the future, please
use 'weights' instead.
  warnings.warn(
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\torchvision\models\_utils.py:223: UserWarning: Arguments other than a
weight enum or `None` for 'weights' are deprecated since 0.13 and may be removed
in the future. The current behavior is equivalent to passing
`weights=ResNet50_Weights.IMAGENET1K_V1`. You can also use
`weights=ResNet50_Weights.DEFAULT` to get the most up-to-date weights.
  warnings.warn(msg)

Downloading: "https://download.pytorch.org/models/resnet50-0676ba61.pth" to
C:\Users\archa/.cache\torch\hub\checkpoints\resnet50-0676ba61.pth

100%|      | 97.8M/97.8M [00:01<00:00, 53.0MB/s]
```

[254]:
```
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.fc.parameters())
```

[255]:
```
def train_model(model, criterion, optimizer, num_epochs=3):
    for epoch in range(num_epochs):
        print('Epoch {}/{}'.format(epoch+1, num_epochs))
        print('-' * 10)

        for phase in ['train']:
            if phase == 'train':
                model.train()
            else:
                model.eval()

            running_loss = 0.0
            running_corrects = 0

            for inputs, labels in dataloaders[phase]:
                inputs = inputs.to(device)
                labels = labels.to(device)

                outputs = model(inputs)
                loss = criterion(outputs, labels)

                if phase == 'train':
```

```
                    optimizer.zero_grad()
                    loss.backward()
                    optimizer.step()

                _, preds = torch.max(outputs, 1)
                running_loss += loss.item() * inputs.size(0)
                running_corrects += torch.sum(preds == labels.data)

            epoch_loss = running_loss / len(image_datasets[phase])
            epoch_acc = running_corrects.double() / len(image_datasets[phase])

            print('{} loss: {:.4f}, acc: {:.4f}'.format(phase,
                                                        epoch_loss,
                                                        epoch_acc))
    return model
```

[256]:
```
model_trained = train_model(model, criterion, optimizer, num_epochs=10)
```

```
Epoch 1/10
----------
train loss: 0.1257, acc: 0.9499
Epoch 2/10
----------
train loss: 0.0830, acc: 0.9690
Epoch 3/10
----------
train loss: 0.0820, acc: 0.9675
Epoch 4/10
----------
train loss: 0.0832, acc: 0.9670
Epoch 5/10
----------
train loss: 0.0774, acc: 0.9700
Epoch 6/10
----------
train loss: 0.0863, acc: 0.9688
Epoch 7/10
----------
train loss: 0.0648, acc: 0.9741
Epoch 8/10
----------
train loss: 0.0608, acc: 0.9753
Epoch 9/10
----------
train loss: 0.0689, acc: 0.9736
Epoch 10/10
----------
train loss: 0.0549, acc: 0.9768
```

[257]:
```
!mkdir models
!mkdir models/pytorch
```

```
The syntax of the command is incorrect.
```

[259]:
```
import torch
torch.save(model_trained.state_dict(), 'models/pytorch/weights.h5')
```

[260]:
```
model = models.resnet50(pretrained=False).to(device)
model.fc = nn.Sequential(
               nn.Linear(2048, 128),
               nn.ReLU(inplace=True),
               nn.Linear(128, 2)).to(device)
model.load_state_dict(torch.load('models/pytorch/weights.h5'))
```

```
e:\Programming\Repositories\Classwork\STAT4210\.venv\Lib\site-
packages\torchvision\models\_utils.py:223: UserWarning: Arguments other than a
weight enum or `None` for 'weights' are deprecated since 0.13 and may be removed
in the future. The current behavior is equivalent to passing `weights=None`.
  warnings.warn(msg)
```

[260]:
```
<All keys matched successfully>
```

[261]:
```
validation_img_paths = ["test_set/test_set/cats/cat.4001.jpg",
                        "test_set/test_set/cats/cat.4003.jpg",
                        "test_set/test_set/dogs/dog.4004.jpg",
                       "test_set/test_set/dogs/dog.4006.jpg"]
img_list = [Image.open(input_path + img_path) for img_path in␣
 ↪validation_img_paths]
```

[262]:
```
validation_batch = torch.stack([data_transforms['validation'](img).to(device)
                                for img in img_list])
```

[263]:
```
pred_logits_tensor = model(validation_batch)
pred_logits_tensor
```

[263]:
```
tensor([[ 3.8026, -4.9477],
        [ 5.4276, -7.5365],
        [-4.3002,  3.0491],
        [-5.8713,  4.5038]], device='cuda:0', grad_fn=<AddmmBackward0>)
```

[264]:
```
pred_probs = F.softmax(pred_logits_tensor, dim=1).cpu().data.numpy()
pred_probs
```

[264]:
```
array([[9.9984157e-01, 1.5838623e-04],
       [9.9999762e-01, 2.3430737e-06],
       [6.4267905e-04, 9.9935728e-01],
       [3.1199233e-05, 9.9996877e-01]], dtype=float32)
```

[265]:
```
fig, axs = plt.subplots(1, len(img_list), figsize=(20, 5))
for i, img in enumerate(img_list):
    ax = axs[i]
    ax.axis('off')
    ax.set_title("{:.0f}% Cat, {:.0f}% Dog".format(100*pred_probs[i,0],
                                                                    ␣
 ↪100*pred_probs[i,1]))
    ax.imshow(img)
```

[266]:
```
labels_input=[]
labels_output=[]
def test_model(model, criterion, optimizer, num_epochs=1):
    for phase in ['validation']:
        model.eval()

        running_loss = 0.0
        running_corrects = 0

        for inputs, labels in dataloaders[phase]:
            inputs = inputs.to(device)
            labels = labels.to(device)
            labels_input.append(labels)

            outputs = model(inputs)

            loss = criterion(outputs, labels)
            _, preds = torch.max(outputs, 1)

            labels_output.append(preds)


test_model(model, criterion, optimizer, num_epochs=1)
```

[267]:
```
y_pred=(torch.cat(labels_output))
y_pred=y_pred.cpu().numpy()
```

```
y_true=(torch.cat(labels_input))
y_true=y_true.cpu().numpy()


from sklearn.metrics import confusion_matrix
confusion_matrix(y_true, y_pred)
```

[267]:
```
array([[1006,    5],
       [  24,  988]])
```

[268]:
```
!pip -q install -U nbconvert[webpdf]
```

```
[notice] A new release of pip is available: 25.3 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

[269]:
```
!python -m playwright install --with-deps chromium > /dev/null
```

```
The system cannot find the path specified.
```

[ ]:
```
!jupyter nbconvert --to pdf "./Stat_4210_Chapter10_Lab.ipynb" --output-dir "./
↪output"
```

```
[NbConvertApp] Converting notebook ./Stat_4210_Chapter10_Lab.ipynb to html
[NbConvertApp] WARNING | Alternative text is missing on 7 image(s).
[NbConvertApp] Writing 2076454 bytes to output\Stat_4210_Chapter10_Lab.html
```